

Assignment 1

Project: Research Infrastructure Costing & Pricing Tool

Client: UWA Research Infrastructure

- **Erika Slavin** — Manager (Research Infrastructure & Partnerships) / Business Development Coordinator
- **Mathew Hall** — Strategic Development Coordinator

Team: CITS5206 Group 13 — Chenxu You, Yichen Zhao, Wenmin Luo, Dai Lam La La, Jaswanth Vericherla

Project resources

Resource	Link
GitHub repository	https://github.com/ChenxuYou/uwa-cits5206-project
MS Teams — client and team group chat	Client & Group 13 · Group Chat
Meeting notes	https://github.com/ChenxuYou/uwa-cits5206-project/tree/main/docs/meetings
Requirements, user stories, architecture	https://github.com/ChenxuYou/uwa-cits5206-project/tree/main/docs/spec
Project plan, risk register, skills audit	https://github.com/ChenxuYou/uwa-cits5206-project/tree/main/docs/project
Client communication history	https://github.com/ChenxuYou/uwa-cits5206-project/tree/main/docs/client/communication-history

1. Problem Statement — what we are building, and why

1.1 What the client does

UWA operates **research infrastructure**: electron microscopes, a human MRI scanner, radio astronomy telescopes, drones that assess plant phenotypes. All of it is expensive to buy and expensive to run, so part of the running cost is recovered from the researchers who buy time on it — who in turn budget for that cost in their grant applications.

Two constraints shape everything that follows, and the client stated both in the first meeting:

- **The money is public.** Much of this equipment is paid for with state and federal funding, so how a price was arrived at has to be explainable, defensible and disclosable under FOI.
- **The method has to be the same everywhere.** Every platform runs differently and is funded differently, but the protocol applied to all of them must be identical — *"apply the same logic for everything."*

The goal is **sustainability, not profit**: charge enough per hour, day or sample that the capability continues to be fundable, and no more.

The method itself already exists and is sound. UWA Research Infrastructure has a costing and pricing guide and a working calculator: total the operating costs, deduct non-variable income, establish annual capacity, forecast realistic utilisation, and divide to obtain three minimum sustainable charge-out rates — UWA researcher, Australian publicly-funded researcher (APFR, with the University's 1.35 indirect cost recovery), and commercial. **The problem is not the method. The problem is the tool the method lives in.**

1.2 What happens today, and why it hurts

The calculator is an eight-sheet Excel workbook: three process sheets — costs, capacity, rates — over a background data layer of salary, usage and competitor-pricing tables. Input cells are yellow, formula cells are blue, and the client is explicit that only the yellow cells are asked of the user.

The client's own verdict on it, unprompted, in the first ten minutes of the kickoff:

Our problem with this is it's functional, but nobody can really use it because it's easy to break.

— UWA Research Infrastructure, 29 July 2026

Four things make that a serious operational problem rather than an inconvenience:

The person doing the work is not a spreadsheet person. The user is a *platform custodian* — the academic or professional staff member who manages the instrument, its bookings, its maintenance and its training. The client described them precisely: *"I do microscopes, I'm really good at microscopes. This kind of stuff is administrative work for them. They don't always love doing it."* They are the right person to forecast utilisation, because they know how popular their equipment is; they are the wrong person to be handed a live formula grid.

The exercise runs once every three to five years. Nobody builds muscle memory for a task they perform twice a decade. Every cycle, a custodian meets the workbook effectively for the first time — and the workbook does not look as complex as it is.

Sharing the file hands over control of the logic. The moment the workbook is emailed to a custodian, that custodian can delete a formula, overwrite a blue cell, or drag a fill handle one column too far. Nothing warns them. Nothing warns anyone afterwards either: the sheet still returns a number, and a number is what gets published as a price.

A single keystroke changes a published rate, silently. The client's own illustration was a maintenance contract typed as **\$200,000 instead of \$20,000** — one extra zero, no challenge, and a charge-out rate that is wrong for the next three years.

1.3 Why the answer has to be a different kind of tool

A better spreadsheet does not fix this, because the fragility is a property of the medium rather than of any particular file. A workbook emailed to a custodian is a workbook whose **formulas** have been emailed to a custodian: logic and data arrive in the same object, with the same permissions, and nothing distinguishes a cell the user is meant to fill from a cell that computes. The client described the consequence directly — *"they have the control to remove formulas, to break things ... it's not foolproof."*

Two properties follow, and between them they are why a rate cannot currently be defended with full confidence:

- **Mistakes are silent.** A spreadsheet has no way to say *"this looks wrong."* A cleared formula, a reference dragged one column too far, or a figure typed with one extra zero all produce a plausible number — and a plausible number is what gets published as a price for the next three to five years.
- **Nothing records how the number was reached.** The workbook holds the current state, not the reasoning behind it. When someone asks in year two why the rate is what it is, the file cannot answer, and the person who built it may no longer be in the role.

Where the guide and the calculator diverge, we ask rather than assume. Working through the client's material we found one place where their costing guide and their calculator produce different commercial rates — the guide applies the indirect cost recovery to the full operating cost, the calculator deducts some grant income first. Rather than pick one, we put it to the client, and on **20 August 2026** they answered in writing: *"Guide governs. Where a discrepancy occurs, we'd appreciate if these can be flagged to us for our knowledge and guidance."* The tool therefore implements the guide, and shows both figures wherever a new number is compared with an existing one, so any difference is visible and explainable rather than silent.

A full reconciliation of the workbook's arithmetic is deliberately out of scope at this stage. It is work for a later cycle, and for a good reason: the engine is the thing you reconcile *against*.

Once the guide's method is implemented and tested, comparing it with the workbook is a matter of running both over the same inputs — cheap, repeatable, and evidence we can hand the client. Doing it now would mean doing it by hand, and then doing it again. The client has asked to be told what we find as we find it, and that is how it will reach them.

1.4 Why the client wants software

Because the number has to survive the question. The client put the requirement in one sentence, and it is the sentence we are building against:

If someone comes to us and says "why does it cost \$50 an hour for me to use?", we want to be able to say: well, it costs \$100,000 a year to run, this is how many hours a year it's going to be used, we divide one by the other — \$50 an hour. That's the reason we charge that price.

And the failure mode they are trying to avoid:

We can't say "we don't know, it's \$50 an hour because we just picked that number". That doesn't sound very good and that doesn't inspire confidence in the people that are using our services.

So the deliverable is not a calculator. It is a **defensible record** — the inputs, the workings, the resulting rates and the custodian's reasoning, sealed, filed, and readable in three years' time when the next cycle opens and someone asks what changed and why.

1.5 What we are building

A guided web application that collects the yellow cells, keeps the blue cells on the server where nobody can reach them, and produces a sealed, retrievable record explaining why a given rate was set.

Every design decision below answers a specific failure above:

The pain today	What the software does instead
A user can delete or overwrite a formula	The calculation runs server-side and is never sent to the browser . There is no formula to break and no way to break the tool by using it
A reference dragged one column too far	Capabilities are a first-class collection and every aggregate iterates that collection . There is no cell range to mis-type, so a total can never be summed over a different set from the figures it is compared against
A cost entered and then silently dropped from a total	Totals are derived from the lines actually entered, not from a hand-written range. If the tool asks for a figure, that figure is counted
\$200,000 typed for \$20,000; a utilisation figure larger than capacity	Mandatory fields with type and range validation , enforced on the server and challenged at the point of entry
A once-in-five-years task, met cold	A sequential guided flow — costs, then capacity and utilisation, then rates — that prompts rather than presents a grid, and can be saved and resumed
A calculated rate nobody would actually charge	The custodian proposes their own rounded rates , sees the resulting surplus or deficit immediately, and records why in free-text justification alongside the numbers
No record of how a price was set	On submit the record is sealed — inputs, calculated rates, proposed rates and justifications become immutable — and exported as a PDF that shows the workings , filed by the custodian into UWA's records system
No way to set this cycle against the last	Past sealed records are listed and reopened ; a new cycle supersedes its predecessor by reference and nothing is ever overwritten or deleted
Anonymous edits	Users sign in ; every record carries who created, submitted and sealed it

1.6 Key deliverables for the MVP

The MVP is a **complete vertical slice** for one platform and its capabilities — a narrow path that works end to end, in preference to three-quarters of every feature. That slice is eighteen Must stories, 110 points, covering requirements **F1 – F13**, **F15** and **F22** in [requirements.md §7](#).

#	Deliverable	What it means
D1	Server-side calculation engine	A pure, versioned, unit-tested module: costs and income in, three rates per capability out, with the intermediate figures behind each. Decimal arithmetic throughout; no I/O, no browser access
D2	Guided three-section workflow	Costs (capability- and platform-level, plus the four income lines) → capacity and forecast utilisation → rates. Save and resume; change any input and see the effect before committing

#	Deliverable	What it means
D3	Validation and identity	Type and range validation server-side on every field, forecast utilisation mandatory and visibly distinct from capacity; authenticated sign-in with custodian, delegated-authority and administrator roles
D4	Proposed rates and balance	Custodian-entered rounded rates, the surplus or deficit that follows, and justification text against every figure
D5	The sealed record	One-way seal on submit; an immutable snapshot with an integrity hash; a PDF carrying the workings as well as the outputs , for filing into Content Manager (TRIM)
D6	Retrieval and supersession	Past sealed records listed and reopened; a new cycle references the one it replaces; validity period 3–5 years, capability dependent
D7	Proof of correctness	An automated golden-file test reproducing the client's own worked example — \$150,000 operating cost, \$20,000 UWA in-kind, \$30,000 State support, 1,000 forecast hours → \$100.00 / \$162.00 / \$202.50 per hour — to the cent , written before any screen exists

The demonstration that proves it. A custodian signs in, starts a cycle for a seven-capability microscopy platform, enters costs and income, builds capacity, forecasts utilisation, sees three calculated rates for every capability, proposes round numbers, sees the resulting balance, justifies it, seals the record and exports the PDF. A second person opens that record and finds the answer to "*why does it cost \$50 an hour?*" — with every figure behind it.

Deliberately out of scope, because naming these is what makes the rest deliverable in one semester: integration with UWA finance, HR or booking systems; billing or invoicing of actual usage; a researcher-facing price lookup; migration of historical spreadsheets; multi-institution operation.

Attempted only if the slice is complete: salary pre-fill from staff levels, in-tool approval routing, a replacement reserve, a benchmarking record, a cross-cycle dashboard, and generation of the price-change communication.

Where the guide and the calculator disagree, the guide governs — confirmed by the client in writing on 20 August 2026. The tool implements the guide's method, shows both figures wherever a new number is compared with an existing one, and reports any divergence to the client as we encounter it.

This scope — what is in, what is out, and what is stretch-only — was **signed by the client on 20 August 2026**. The evidence is in §2.

2. Client Communication and MVP Agreement

Clients: Mathew Hall and Erika Slavin

Meeting 1 — 29 July 2026

- Clients introduced the project and its requirements
- Gave the team its initial understanding of the costing and pricing problem
- Team set up a Teams group chat with Mathew and Erika to keep communication ongoing

Supporting material received — 11 August 2026 (via email from Mathew)

- **CONTEXT – Costing & Pricing Research Infrastructure:** the guide itself — six-step method, the worked example used as our automated test case (\$150,000 costs, \$20,000 UWA in-kind, \$30,000 WA Government, 1,000 hours → \$100.00 / \$162.00 / \$202.50 per hour), and the standard UWA capacity baseline (230 days × 7.5 hrs ≈ 1,725 hrs/year)
- **TEMPLATE – RIC Cost Calculator:** the live spreadsheet the tool replaces, populated with real Cytometry platform data across eight capabilities — used to cross-check the guide and surface where the two disagreed
- **EXAMPLE – Project Costing Template:** a live UWA tool costing individual projects (staffing + other costs, editable/exportable output) — precedent for the guided-input pattern, though it costs a project rather than a platform and has no capacity/utilisation step
- **EXAMPLE – UniSuper Calculator:** an external retirement calculator with a pre-filled, editable goal and a clear "meet/fall short/exceed target" output — precedent for showing a proposed rate against a calculated minimum

Scope drafting and sign-off request — 17 August 2026

- Compared the guide against the calculator's actual formulas and found several disagreements, notably in commercial-rate treatment
- Drafted and sent two documents by email:
 - **Project Scope Summary and Sign-off** — what we're building, what we're not, and two confirmations (scope and IP ownership)
 - **Five Open Questions** — each with a default answer, so nothing was blocked while waiting for a response

Meeting 2 — 20 August 2026

- Demonstrated the team's understanding of the scope
- Walked through the five open questions with the clients
- Clients gave positive feedback and returned both documents with responses

Client responses to the five open questions

#	Question	Client answer
1	Who is allowed to see and approve a record?	Three-role model (custodian, delegated authority, administrator) confirmed. Approval must be in the core, not stretch — a nominated approver, granted by the administrator, approves records. If SSO is added later, link to HR for staff roles.
2	How do multi-year cycles work?	No amendment needed. Errors found after sealing are handled by raising a new record next cycle; old records stay as-is.
3	What should the sealed record look like, and where does it get filed?	No fixed template required. Export should include a plain-language summary of what changed from the previous record.
4	Guide vs. calculator on commercial rates	No comment — default confirmed: tool follows the guide (no income deducted before the 1.35 uplift).
5	Guide vs. calculator generally	Client suggested generating both a guide-based and calculator-based version of a record for comparison, not just documenting the difference in text.

Sign-off

- Signed by **Mathew Hall, Strategic Development Coordinator**, 20 August 2026
- Both confirmations checked: (1) scope as described, correct subject to the approval-workflow change above; (2) IP/ownership — UWA owns the method, team owns the code, tool IP held jointly

MVP agreement

Minimum end-to-end path through the guided costing tool, updated per the client's answers:

- Custodian signs in, starts a cycle, enters costs/income for a capability
 - Builds capacity from standard baselines, forecasts utilisation
 - Sees the three minimum sustainable rates with underlying figures shown
 - Proposes a rounded rate, sees surplus/deficit, records justification
 - **Submits to a nominated approver** (Q1) rather than sealing directly
 - Once approved, record is sealed and exported with a **plain-language change summary** (Q3)
 - One platform, one capability, one full cycle — engine proven against the guide's worked example and the live Cytometry data before the interface is built out
-

3. Project Management and Plans

We run **one-week sprints on a Saturday-to-Saturday cycle**, each ending in a client-facing increment. The plan below covers every week from the day after this submission to the final deliverable on **13 October 2026**. Every task has one named owner and one date; the full backlog lives on the GitHub Projects board.

3.1 Milestones

#	Milestone	Date	Done when
M0	Assignment 1 submitted	25 Aug 2026	One PDF uploaded by one member; every linked resource open to the facilitator
M1	Engine provably correct	4 Sep 2026	The client's worked example reproduces to the cent, as a CI merge gate. Written before any screen
M2	Guided flow, validated server-side	11 Sep 2026	Costs, income, capacity and mandatory forecast utilisation captured and validated
M3	Rates, proposed rates and balance	18 Sep 2026	Three rates per capability with the figures behind each; proposed rates and the resulting surplus or deficit
M4	Vertical slice complete	25 Sep 2026	Sign in → create cycle → enter inputs → see rates → propose → justify → seal → export PDF → reopen
M5	Staging live, client using it	2 Oct 2026	Deployed over HTTPS, seeded credentials replaced, the client reaches it unaccompanied
M6	Release candidate, feature freeze	9 Oct 2026	Critical fixes only; full regression pass; evidence pack assembled
M7	Final release and handover	13 Oct 2026	Tagged release deployed, handover notes written so UWA can rehost, final report submitted

If a milestone is at risk, the stretch list gives way — never the vertical slice. The cut order is fixed in advance: dashboard → price-change communication → benchmarking record → replacement reserve → salary pre-fill → in-tool approval routing.

3.2 Who owns what

Five members, five distinct areas. The split follows the structure agreed at the team meeting of 15 August 2026, and is deliberately drawn by *role* rather than by document, so that nobody's contribution is defined only by which file they happened to edit.

Area	Owner	Responsibility
Client liaison and sprint coordination	Yichen Zhao	Owns all client contact — one voice, so the client is never chased twice for the same thing. Books meetings, sends the weekly demo summary, records feedback as issues, and maintains the decision log

Area	Owner	Responsibility
Development — application and workflow	Chenxu You	Owns the Razor Pages workflow, authentication and approvals, the deployment pipeline, and release management
Development — calculation engine and data	Wenmin Luo	Owns the server-side calculation engine, the data model and migrations, the CI configuration, and the server/hosting setup
Costing logic and risk	Dai Lam La La	Owns the reading of the client's guide, the assumptions log, the risk register, and anything raised with the client where their material needs interpretation
Test scenarios, verification and documentation	Jaswanth Vericherla	Owns the test scenarios and golden-file fixtures, verification of each increment against the client's worked example, link and access checking, and meeting minutes

Development and testing are **separate owners on purpose**: the person who writes a calculation does not sign off its arithmetic. Everyone reviews pull requests; nothing merges on its author's approval alone.

3.3 Weekly cadence

When	What happens	Owner
Saturday evening	Team meeting: review the sprint, reprioritise the backlog, assign owners and dates for the next week. Minuted and committed within 24 hours	Whole team; minutes by Jaswanth Vericherla
Monday–Thursday	Build and test in parallel, so every change is verified before it is shown to anyone	Chenxu You, Wenmin Luo (build); Jaswanth Vericherla (test)
Wednesday	Client touchpoint — demo the current increment and capture feedback. Wednesdays are the client's stated preference	Yichen Zhao
Thursday	Feedback becomes issues on the board, with owners, before the next planning meeting	Yichen Zhao, Dai Lam La La
From 28 September	Final report drafted alongside the build, not after it, so it tracks the project's actual state	Dai Lam La La, Yichen Zhao

3.4 Project tools and the evidence they produce

Every link below is a live resource the facilitator can open.

Tool / artefact	Purpose	Where
GitHub repository	Single source of truth for specification, plans and code	link

Tool / artefact	Purpose	Where
GitHub Issues	Every task carries one owner and one due date	link
GitHub Projects board	Sprint planning and work in progress — Backlog · In Progress · Review · Done, populated from the eighteen Must stories	link
Pull requests	Code review and ownership; nothing merges unreviewed	link
GitHub Actions CI	Build, test and dependency scan on every push and pull request; the engine tests are the merge gate	link
Meeting notes	Decisions and client feedback, numbered and traceable across meetings	link
Project plan	Milestones, sprints and story assignment	link
Risk register	Likelihood × impact × mitigation × trigger × owner	link
Skills audit	Where our gaps are, and what is done about each	link
Decision records	Architecture and process decisions, with the evidence behind them	link
Client communication history	One folder per exchange — what was sent, what came back	link
MS Teams — client and team group chat	Asynchronous questions to the client, recordings and files that do not belong in a public repository	Client & Group 13 · Group Chat

3.5 Deployment plan

Deployment is the largest question still open from the 15 August meeting, so it is treated as a milestone with a decision date rather than as something that happens at the end.

Stage	What we do	Owner	By
Decide where it runs	Put the three options to the client — a UWA-hosted VM, the UWA domain already shared with us, or a team-provisioned cloud host — and confirm who administers it and whether sign-in must use UWA accounts	Yichen Zhao	9 Sep 2026
Provision and access	Obtain server access, create the deployment target, record credentials handling	Wenmin Luo	18 Sep 2026
Pipeline	Extend CI to CD: build, test, publish, deploy on merge to <code>main</code> , with a documented rollback	Chenxu You	25 Sep 2026

Stage	What we do	Owner	By
DNS and reverse proxy	Configure DNS and, if required, Nginx as reverse proxy and TLS termination	Wenmin Luo, Chenxu You	30 Sep 2026
Deployment testing	Exercise build, release, rollback and reachability end to end; confirm the app runs over HTTPS with the seed accounts replaced	Jaswanth Vericherla	2 Oct 2026
Staging sign-off	Confirm the deployed instance is stable enough for the client to use unaccompanied	Whole team	5 Oct 2026
Final release and handover	Tag the release, deploy it, and package handover notes so UWA can rehost	Chenxu You, Wenmin Luo	13 Oct 2026

3.6 Week by week

Week commencing	Sprint goal	Yichen Zhao	Chenxu You	Wenmin Luo	Dai Lam La La	Jaswanth Vericherla	Output
24 Aug	Turn the signed scope into a working backlog	Close action A14 (is in-tool approval required in the core?); book the September touchpoints	Populate the board from the eighteen Must stories; write ADR-001	Extract the calculation into a testable engine module	Requirement IDs and estimates for the two items the client added on 20 August	Write the golden-file fixture from the guide's worked example	Board populated, ADR-001, engine module, backlog estimated
31 Aug	The engine is provably right before anything is built on it	Weekly summary; confirm sprint priorities with the client	Move k , capacity baselines and categories into versioned configuration	Decimal arithmetic end to end; divide-by-zero guard; aggregates iterate capabilities	Confirm the guide's Step 3 formulas line by line against the engine contract	Golden-file test passes to the cent; property tests on the boundaries	M1 — engine passing the client's worked example, CI gate green
7 Sep	The guided flow, validated server-side	Client demo; feedback into issues	Costs and income screens with server-side validation	Capacity, deductions and mandatory forecast utilisation	Document the assumption behind each validation rule	Scenario tests for the extra-zero typo and for utilisation entered above capacity	M2 — sections 1 and 2 of the flow working; validation evidence
14 Sep	Rates, proposed rates and balance	Put the deployment options to the client and get a decision	Rates screen: three rates per capability with the figures behind each	Proposed rates, variance and platform roll-up	Check the roll-up against a worked capability from the client's material	Regression suite across all seven capabilities	M3 — section 3 working; deployment decision recorded
21 Sep	Seal, export, retrieve	Confirm the PDF layout with the client (workings included)	Seal transition, immutable snapshot, integrity hash	Provision the server; PDF generation showing the workings	Check the exported record answers " <i>why does it cost \$50 an hour?</i> "	Test retrieval, supersession and the read-only path on a sealed record	M4 — vertical slice complete, end to end
28 Sep	Deploy to staging and harden	Coordinate DNS; invite the client to use staging unaccompanied	CI to CD, rollback tested, demo credentials replaced	DNS and reverse proxy; HTTPS and secrets handling	Start the final report against the project's actual state	Deployment test log; accessibility and link checks	M5 — working staging instance the client can reach
5 Oct	Stabilise, and only stabilise	Draft the client-communication and deployment sections	Critical fixes only; feature freeze	Critical fixes only; performance and error handling	Assemble the evidence pack	Full regression pass; archive test evidence	M6 — release candidate, report draft, evidence pack

Week commencing	Sprint goal	Yichen Zhao	Chenxu You	Wenmin Luo	Dai Lam La La	Jaswanth Vericherla	Output
12 Oct	Final release and handover	Finalise the report; confirm handover with the client	Tag and deploy the final release	Handover notes so UWA can rehost	Final read-through by someone who did not write it	Verify the final build; check every link resolves	M7 — final release, final report, handover pack

Where the next cycle picks up. Two things are consciously left for the period after this build rather than squeezed into it: reconciling the client's existing calculator against the engine line by line, and the integration work — UWA single sign-on, HR-sourced staff roles, and writing records into Content Manager (TRIM) — that the signed scope defers. Both become far cheaper once the tool exists, and both are recorded so that neither is quietly forgotten.

4. Risk and Technology Assessment

4.1 Skills and resources

The team is five people for eleven weeks. DongSheng Li withdrew from the unit on 27 July 2026 and his work was redistributed; the roster in [team.md](#) has been five since. That is the resource, and every plan in §3 is sized against it rather than against the six we started with.

We ran a **skills audit** across the six competencies the project actually needs, self-rated 1–5, and recorded it in [skills-audit.md](#). Two things came out of it. The first is that the team's depth is concentrated in C#/ .NET and server-side web work, and is thin in JavaScript frameworks — which is the single largest input to the technology decision in §4.2. The second is a list of gaps that are real, and each of them carries a countermeasure rather than an intention:

Gap	Why it threatens this project	How it is addressed
Automated testing discipline — nobody has built a project around a test gate before	The whole value proposition is <i>the arithmetic can be trusted</i> . Untested code makes it worthless	The golden-file test against the client's worked example is written before the screens (D7) and is a CI merge gate. Jaswanth Vericherla owns test scenarios independently of the developers who write the code
Deployment and operations — no member has taken an application to a running server with TLS, DNS and a rollback path	Criterion: the client must be able to <i>use</i> it. A demo on <code>localhost</code> is not a handover	Deployment is a milestone with dates and owners (§3.5), not an end-of-semester task. Staging is live by 28 September so the client has two weeks to use it

Gap	Why it threatens this project	How it is addressed
Turning a spreadsheet process into a workflow — translating a lived business method into software is where projects of this shape fail	Misreading the method produces a tool that is confidently wrong	Written source-precedence rule (client's written answers → guide → calculator → our minutes); every requirement carries its source marker; one member (Dai Lam La La) owns the reading of the client's guide rather than five people forming five opinions
Financial/costing domain knowledge — none of us is an accountant	Indirect cost recovery, non-variable income and FEC are not intuitive	The client stated explicitly that we do not need to master the reasoning, only implement it faithfully. We ask rather than infer: five questions went to the client on 17 August and all five came back answered
JavaScript / SPA frameworks — unproven depth	Would be fatal if we had chosen a SPA architecture	Addressed by <i>not choosing that architecture</i> — see §4.2. This is a gap we designed around rather than one we have to close
Bus factor on the calculation engine — two of five write production code	One person unavailable in week 9 stalls the build	The engine is a plain library with no framework dependencies; both developers pair on it; all pull requests are reviewed by a second member; no merge on the author's own approval

4.2 Technology assessment

What the architecture has to achieve

Six drivers, each traceable to a specific failure in §1, decide the stack. Everything else is preference.

Driver	Comes from
The user must not be able to reach the calculation	Sharing a workbook shares its formulas — the client's central complaint
A total must always be summed over the same set as the figures it is compared against	A spreadsheet range is typed by hand and can silently span the wrong columns
Money arithmetic must be exact and reproducible	Public money, FOI, a record read three years later. Binary floating point is not defensible in an FOI response
A record must reproduce its own figures years later	<i>"Open the old record in three years and see why"</i>
A returning user must succeed without training	Once every 3–5 years, by a microscope specialist

Driver	Comes from
We must ship something complete	One semester, five members, mixed skills

Options assessed

Six options were considered; five were presented to the facilitator at the checkpoint of 5 August 2026, and all are recorded in

[architecture.md §8](#).

Option	Shape	Verdict
A	Client-side SPA, no backend	Rejected as the product. Puts the calculation in JavaScript the user can read and modify — precisely the defect we were engaged to remove. Valuable only as a throwaway flow prototype
B	SPA (React/Vue) + REST API + PostgreSQL	Considered, not chosen. Meets every requirement and has the highest ceiling, but two codebases, two build chains and auth across the boundary. It is the option most likely to spend the semester on plumbing, and it leans hardest on the skill the audit shows we are thinnest in
C	Server-rendered monolith — Django + HTMX + PostgreSQL	Considered, not chosen. The right <i>shape</i> : one codebase, framework-provided auth, ORM, CSRF and form validation, and nowhere for calculation logic to leak to. Rejected only on language fit — see below
D	Microsoft Power Platform	Rejected. The failure we are replacing is a spreadsheet-adjacent low-code tool; replacing it with another one is not a fix. Logic sits in a proprietary environment where golden-file testing is awkward, and licensing and tenancy access are outside our control
E	Build inside existing UWA systems	Deferred by the client, in their own words: <i>"integration would be ideal, but I think that might be a bit challenging"</i> — a functional standalone website first. Not scored, because what rules it out is availability, not merit
F	Server-rendered monolith — ASP.NET Core Razor Pages + EF Core	Chosen. Option C's architecture in the language the team can actually move fastest in

The decision, and how we reached it

We chose **F: ASP.NET Core Razor Pages with Entity Framework Core**, recorded as [ADR-001](#).

The reasoning has two halves, and we state the second one plainly because the order in which things happened matters:

Architecture first. The comparison in `architecture.md` §8 scored the server-rendered monolith (C) ahead of the SPA (B), driven by two criteria weighted at 4 and 5 — probability of shipping something complete, and fit to the team's current skills. A server-rendered application makes the hardest requirement *structural rather than disciplinary*: if the browser only ever receives rendered HTML, there is no formula for a user to reach, and no amount of carelessness later can reintroduce one.

Then language, decided by evidence rather than by assertion. C's language was Python; ours is C#. Rather than argue about it, we **built a timeboxed spike** — the working application now in `src/`, covering sign-in, roles, the guided RIC workflow, the rate calculation, an approval queue, notifications, and sealed snapshots with a SHA-256 integrity hash. It was running in time for the client meeting of 20 August. **That spike is the skills evidence:** it demonstrates, rather than claims, that the team's two developers can deliver a complete vertical slice in this stack within a sprint.

Four properties of .NET map onto the drivers above unusually directly:

Driver	What ASP.NET Core gives us
Exact money arithmetic	<code>decimal</code> is a native 128-bit base-10 type in C#, not a library opt-in. <code>0.1 + 0.2 == 0.3</code> holds, and it holds by default rather than by remembering to import something. JavaScript has no decimal type at all; Python requires <code>Decimal</code> discipline at every boundary
Logic out of the user's reach	Razor Pages render server-side. There is no client bundle for the calculation to be compiled into
Aggregates that cannot span the wrong set	LINQ over a strongly-typed capability collection. There is no cell range to mis-type, and the compiler rejects a mismatched aggregate
Baseline security for free	Antiforgery tokens on by default, Razor auto-escaping against XSS, EF Core parameterising every query, ASP.NET Core Identity password hashing, cookie authentication with role policies enforced in page conventions rather than in navigation

The honest part. The decision record was written **after** the spike, not before it. We built to learn, and the ADR documents what we learned; presenting it the other way round would be a tidier story and a false one. What we will not do is leave the repository arguing for one stack while shipping another — ADR-001 exists precisely to close that gap, and `architecture.md` §8 now carries Option F alongside the five it already assessed.

What the choice costs us, and what we do about it

Trade-off	Mitigation
.NET 10 is very new , so there is a smaller body of worked examples and some libraries lag	The SDK version is pinned in CI. The dependency surface is deliberately tiny — one NuGet package today. Nothing in the design needs a bleeding-edge feature

Trade-off	Mitigation
Less interactive polish than a SPA for the scenario modeller	Acceptable: the client asked for prompts and boxes, not a live dashboard. If richer interaction is needed, it is added as progressive enhancement on top of working pages
SQLite is the development store; production may want PostgreSQL	EF Core makes the provider a one-line change, and no raw SQL is written anywhere. The decision is deliberately deferred to the deployment decision on 9 September, when we know where it runs
Framework knowledge concentrated in two members	The calculation engine is a plain class library with no ASP.NET dependency — portable, and reviewable by anyone who can read the formulas

Stated fallback trigger. If a working end-to-end slice — sign in → create cycle → enter inputs → see rates → seal → export — is not running by the end of week 8, we cut stretch scope rather than change stack. The engine, the data model and the tests are deliberately isolated so that a stack change *could* be survived; but changing stack at that point would be the wrong answer, and saying so in advance stops it being an option we reach for under pressure.

4.3 Risk assessment

Risks are held in a single register, [risks.md](#), reviewed at the Saturday meeting. Every row has a named owner, a mitigation, and a **trigger** — the observable event that means the mitigation is no longer optional. The profile is dominated by four areas: **correctness, scope, security and delivery**.

Correctness is the risk that matters most, and it is unusual in that we can point at it happening already: the client's current tool produces a surplus that does not exist. The threat to this project is not an attacker — it is *an incorrect or undetectably altered rate*, published for three years, defended in an FOI response. Every structural decision in §4.2 is aimed there.

Risk register (extract)

#	Risk	L	I	Mitigation	Trigger	Owner
R1	A calculation defect reaches a published rate	Low	Critical	Pure server-side engine; decimal arithmetic; aggregates iterate a typed collection; golden-file test against the client's worked example as a CI merge gate; independent test owner	Any engine test failing on <code>main</code> , or any figure differing from the guide by more than a cent	Wenmin Luo
R2	The client's method is misunderstood and we build the wrong thing correctly	Medium	High	Written source-precedence rule; every requirement carries <code>[C]</code> / <code>[G]</code> / <code>[W]</code> / <code>[K]</code> ; one owner for the reconciliation; questions batched and asked rather than assumed	Two documents disagreeing with no recorded decision	Dai Lam La La

#	Risk	L	I	Mitigation	Trigger	Owner
R3	Scope grows past the semester	Medium	High	The vertical slice is the commitment; the stretch list is explicitly "intentions, not commitments", and the cut order is stated in advance (§3.5)	Any stretch item started before the slice is complete	Whole team
R4	Deployment is left to the end and does not happen	Medium	High	Deployment is a milestone with owners and dates (§3.4); staging live 28 September; client uses it unaccompanied for two weeks	The 9 September hosting decision slipping	Chenxu You
R5	Client latency blocks a decision	Low	Medium	One liaison; questions batched; every open question carries a default we will use if no answer comes . All five went out on 17 August and all five came back — none was defaulted	Any question open for more than seven days	Yichen Zhao
R6	Key member unavailable — five people, one already withdrawn	Medium	High	Two owners for every critical area; PR review by a second member; engine kept framework-independent; work redistributed and documented when DongSheng Li withdrew	Any area with a single owner for more than one sprint	Whole team
R7	Cybersecurity — injection, XSS, CSRF, mass assignment	Low	High	EF Core parameterised queries, no string-built SQL; Razor auto-escaping, including justification free text on PDF render; antiforgery tokens on all state-changing requests; explicit view models, never binding a request body onto an entity	Any raw SQL, any <code>[BindNever]</code> removed, any user text rendered unencoded	Wenmin Luo
R8	Cybersecurity — broken access control . Records are UWA-internal and FOI-subject; a custodian must not see another platform's cycle	Medium	High	Authorisation enforced server-side in page policies and POST handlers , not by hiding navigation; roles as page conventions; every handler re-checks ownership	Any authorisation check found only in a view	Chenxu You
R9	Cybersecurity — credentials or client data committed to a public repository	Low	High	<code>.gitignore</code> §1 (client material), §4 (keys and secrets), §5 (local databases); secrets from environment variables; demo credentials replaced before staging ; dependency scanning in CI	Any secret, <code>.db</code> file or client spreadsheet appearing in a diff	Chenxu You
R10	Tampering with a sealed record at the database level	Low	High	One-way seal transition; no route accepts a write to a sealed record; immutable JSON snapshot with a SHA-256 integrity hash ; recomputation on read forbidden; append-only audit entries	Any hash mismatch on verification	Wenmin Luo

#	Risk	L	I	Mitigation	Trigger	Owner
R11	.NET 10 immaturity — a dependency or SDK regression blocks the build	Low	Medium	SDK version pinned in CI; minimal dependency surface — one direct NuGet package today	A build failing on the toolchain rather than on our code	Wenmin Luo
R12	A dependency vulnerability reaches production	Low	Medium	CI scans every push and pull request; High and Critical advisories fail the build unless assessed and recorded with a written reason and a review date. This one has already fired: CVE-2025-6965 in SQLite, reached through EF Core's provider, has <i>no patched version at all</i> — so it is accepted on the evidence that exploiting it requires crafted SQL and we issue none, and it is scheduled for review with the hosting decision	Any High or Critical advisory not already assessed	Wenmin Luo
R13	IP or licensing dispute over a jointly-owned deliverable	Low	Medium	Ownership position confirmed in writing by the client on 20 August; NOTICE grants UWA perpetual internal-use permission and reserves portfolio rights for each author; no open-source licence granted until handover, because one joint owner cannot grant one alone	Any request to relicense or redistribute before handover	Chenxu You

Risks we have already realised and closed

The clearest evidence that this register is used rather than merely written is that three risks on it have already fired, been caught, and been closed — two of them by our own review of our own documents.

What happened	How it was caught	Outcome
An MIT licence was published in an early commit for a project whose IP is jointly held	Repository review	Rewritten out; NOTICE now states the interim all-rights-reserved position and the reason for it
A draft of the client sign-off document asked the client to confirm that the overarching IP was UWA's alone — the opposite of the position our own NOTICE holds, and the very confirmation that closes our licensing question	Review of our own outbound document, before sending	Corrected before it left the building. The signed version states joint ownership

What happened	How it was caught	Outcome
A set of figures transcribed from the kickoff recording was heading into a test fixture, having appeared in no client document	The written source-precedence rule caught them	Withdrawn from all downstream use; fixtures rebuilt from the client's guide

A team that treats its own documents as things that can be wrong is applying, to itself, exactly the discipline this product exists to enforce on a spreadsheet.